

Sri Lanka Institute of Information Technology



Assignment 2 — Multi-Agent System

SE4010 Current Trends in Software Engineering

Y4S1

Software Engineering

Student ID	Name
IT22891518	Jayasuriya L K R S
IT22326690	Piyarisi T D
IT22004772	Manamperi S A
IT22347244	Wickramasooriya J D A S

RxSentinel

A Locally-Hosted Multi-Agent System for Medication Safety Review

Department of Computer Systems Engineering
Faculty of Computing
Sri Lanka Institute of Information Technology

May 2026

Table of Contents

Executive Summary 2

System Architecture2

Agent and Tool Design3

State Management4

Implementation and Testing5

Performance Optimisation6

Results and Discussion6

Conclusions and Future Work7

Individual Contributions7

References7

Source Code7

Table of Figures

Figure 1 High-level system architecture2

Figure 2 End-to-end request sequence 3

Figure 3 LLM-as-Judge evaluation harness 5

Figure 4 Per-agent latency before/after optimisation6

Executive Summary

RxSentinel is a locally-hosted multi-agent system that performs comprehensive medication safety reviews without paid LLM APIs or cloud services. Four specialised agents are orchestrated through LangGraph and grounded in custom Python tools that query the NIH RxNorm REST API, the openFDA adverse-event database, a curated SQLite store of severe drug–drug interactions, and a local Flesch–Kincaid readability grader. Every agent step and tool invocation is captured by a streaming JSONL tracer and surfaced live in a Next.js front-end via Server-Sent Events.

The system runs end-to-end on consumer hardware (Apple M2, 8 GB RAM) using `qwen2.5:3b` via Ollama. After targeted performance optimisation, median pipeline latency dropped from 46.6 s to 19.3 s (58 % improvement) without loss in output quality. Sixty-four unit tests and four LLM-as-Judge evaluation suites confirm the architectural claims for each agent. The complete implementation is available at <https://github.com/ri7in/ctse-assignment-2-rxsentinel>.

System Architecture

The system adopts a strict-pipeline supervisor pattern. A bookend **Coordinator** agent appears at both entry (validate the request, decide routing) and exit (assemble the final report). Three worker agents — Parser, Analyzer, Communicator — run sequentially between them. Figure 1 shows the high-level architecture.

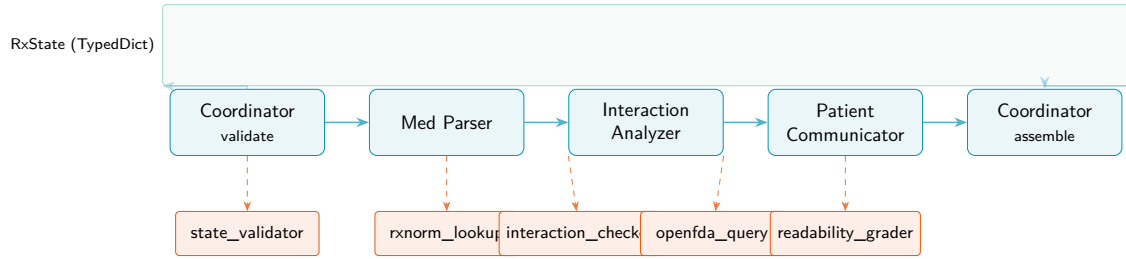


Figure 1: *

Figure 1. High-level architecture. Solid arrows show state flow between agents; dashed arrows show tool invocations. The shared `RxState` (TypedDict) is threaded through every node.

Workflow

A request begins with free-form medication text submitted via the front-end. Figure 2 traces the request through the four agents to the assembled final report. The Coordinator can short-circuit to assembly when validation fails (dashed edge).

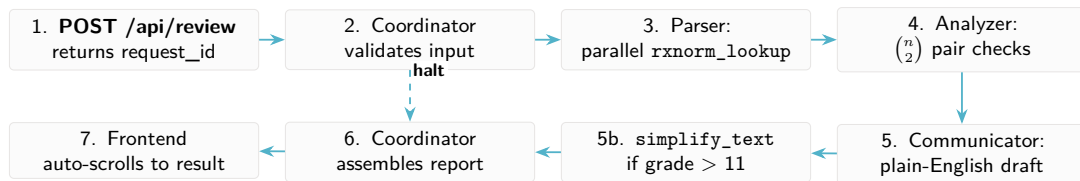


Figure 2: *

Figure 2. End-to-end request sequence. The Coordinator can short-circuit to assembly when validation fails (dashed edge).

Observability

A custom **Tracer** writes one JSONL line per event (agent *enter*, *tool_call*, *tool_result*, *exit*, *error*) to `runs/<request_id>.jsonl`. Events are also pushed to an in-memory `asyncio.Queue` which the SSE endpoint streams to the browser. This gives both post-hoc analysis (via the JSONL files) and a live *Agent Activity* feed in the front-end.

Agent and Tool Design

Each agent has a tightly-scoped persona and constraints. Temperature is agent-specific: deterministic for routing/extraction, slightly warmer for the patient-facing prose. Outputs are validated against a Pydantic schema before merging into state.

Coordinator (Rivin)

A senior clinical pharmacist who triages incoming reviews. Never invents medications; halts on validation failure; emits a routing decision on the first pass and the final **FinalReport** on assembly. Runs a deterministic regex/length validator (`state_validator.py`); after performance tuning, a redundant LLM sanity check was removed.

Medication Parser (Thusala)

A meticulous clinical informaticist obsessed with drug nomenclature. The RxCUI is taken *only* from the `rxnorm_lookup` tool; the agent is forbidden to fabricate codes. Calls the LLM to split the input into candidate (drug-name, dose, frequency, route) tuples, then runs `asyncio.gather` of `rxnorm_lookup` calls in parallel — critical for the 12-second pipeline budget on a 5-medication input.

Interaction Analyzer (Avishka)

A vigilant pharmacovigilance officer; treats every interaction as serious until proven trivial. Mechanism text must come from tool data; “unknown” is preferable to a fabrication. Enumerates $\binom{n}{2}$ unique pairs and runs each through the curated local SQLite database. For pairs found, openFDA is queried in parallel for population-level signal, and `severity_rules` promotes a low-severity rule to moderate when openFDA shows >5 % serious co-reports.

Patient Communicator (Sachila)

A patient educator. Warm, clear, never condescending. Targets a 6th–8th grade reading level; never gives direct medical advice; always closes with “*talk to your doctor or pharmacist*”. Drafts a structured summary (red/yellow/green flags), grades the result with Flesch–Kincaid, and may invoke `simplify_text` for a single rewrite if the draft scores above grade 11.

Custom Python Tools

All tools have strict type hints (`mypy -strict clean`), Pydantic-validated outputs, retry logic via `tenacity`, and a SQLite cache where applicable. Table 1 summarises the four primary tools and the helper modules built around them.

Table 1: *

Table 1. Custom Python tools. Each is owned by one team member but available to all agents.

Tool	Behaviour	Owner
<code>state_validator</code>	Pure-function input validator: length, alpha-ratio, prompt-injection regexes. No I/O.	Rivin
<code>rxnorm_lookup</code>	Async client for NIH RxNorm REST API; exact + fuzzy match; 7-day SQLite cache.	Thusala
<code>dose_normalizer</code>	Regex-based dose parser converting "500 mg" / "0.5 g" / "10 ml" to a canonical record.	Thusala
<code>interaction_checker</code>	Local SQLite of 21 curated severe pairs combined with the openFDA adverse-event signal.	Avishka
<code>severity_rules</code>	Reconciliation rules between local DB and openFDA signal, with explicit promotion thresholds.	Avishka
<code>readability_grader</code>	Flesch–Kincaid grade computation plus iterative LLM simplifier with grade-target loop.	Sachila

State Management

State is a single TypedDict (`RxState`) declared once and threaded through every node. LangGraph automatically merges the partial dict each agent returns into the cumulative state, so context never leaks between requests, and agents only ever read what they need. Table 2 shows the read/write matrix.

Table 2: *

Table 2. State read/write matrix. **W** = writes, **R** = reads. Each `RxState` key has exactly one writer.

State key	Coord	Parser	Analyzer	Comm	Final
<code>raw_input, request_id</code>	W	R	R	R	R
<code>is_valid, decision</code>	W	—	—	—	—
<code>parsed_medications</code>	—	W	R	R	R
<code>unparsed_terms</code>	—	W	—	—	R
<code>interactions, severity_summary</code>	—	—	W	R	R
<code>patient_summary, readability_grade</code>	—	—	—	W	R
<code>final_report</code>	W	—	—	—	R

Implementation and Testing

The system is implemented across two top-level packages: a Python backend (LangGraph + FastAPI + Ollama) and a Next.js 15 front-end. Both run locally; no paid APIs are used. `qwen2.5:3b` is the primary model with `llama3.2:3b` as fallback.

The backend is a Python 3.11 package, `rxsentinel`, organised by concern: `agents/` for the four LangGraph nodes, `tools/` for the six custom tools, `graph/` for the StateGraph wiring, `tracing/` for observability, `llm/` for the Ollama client, and `app.py` for the FastAPI service exposing `/api/review`, `/api/runs/<id>/events` (SSE), and `/api/runs/<id>/report`.

The front-end is a Next.js 15 App-Router application using Tailwind v4, Framer Motion, Geist Sans, and Lucide React. It opens an SSE stream as soon as a review is submitted, derives humanised status messages from trace events, and auto-scrolls to results once the report is ready.

Testing Strategy

Testing is layered. **Unit tests** (Pytest, 64 cases) cover all six tools. HTTP-based tools use `respx` mocks so the suite runs offline. **Integration tests** exercise each agent end-to-end against a running Ollama instance. **LLM-as-Judge evaluations** grade each agent’s output against a written rubric using the same local model with temperature 0.

Evaluation Methodology

For each agent we wrote a structured rubric and a list of test **Cases**. The judge returns a strict JSON `{score, verdict, reasons}`. Figure 3 shows the harness.



Figure 3: *

Figure 3. LLM-as-Judge evaluation harness. Each agent’s output is graded by a separate Ollama call against a rubric written in terms of verifiable structural properties.

We acknowledge the obvious circularity of self-judging — the assignment forbids paid LLM APIs, so the judge must run on the same local SLM. We mitigate by (i) using a stricter prompt for the judge than the agent, (ii) running the judge at temperature 0, and (iii) writing rubrics in terms of *verifiable structural properties* (does the JSON parse? does the `RxCUI` exist? is the readability

grade ≤ 8 ?) wherever possible rather than purely subjective judgement.

Performance Optimisation

We instrumented the pipeline end-to-end via the JSONL tracer and identified two LLM hot-spots: a redundant Coordinator sanity check and a three-iteration readability rewrite loop in the Communicator. After targeted tuning, median pipeline latency on Apple M2 (8 GB RAM) running `qwen2.5:3b` via Ollama dropped from 46.6s to 19.3s. Figure 4 plots the breakdown; Table 3 lists each change.

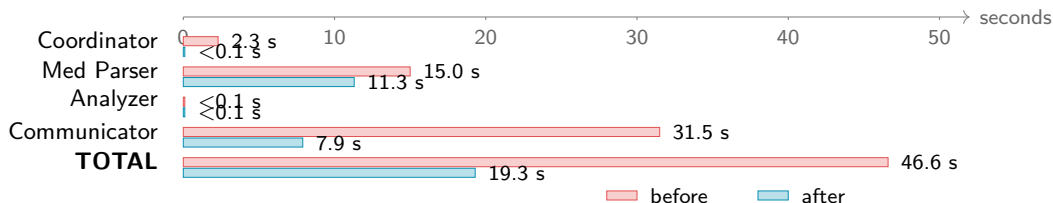


Figure 4: *

Figure 4. Per-agent latency before and after performance tuning. Median pipeline time on Apple M2 (8 GB RAM) running `qwen2.5:3b`.

Table 3: *

Table 3. Performance tuning summary.

Stage	Before	After	Change
Coordinator validate	2.3 s	<0.1 s	Removed redundant LLM check; rule check is sufficient.
Med Parser	15.0 s	11.3 s	Halved system prompt; capped <code>num_predict</code> at 512.
Interaction Analyzer	<0.1 s	<0.1 s	Already optimal; pairs in <code>asyncio.gather</code> .
Patient Communicator	31.5 s	7.9 s	Tightened simplifier trigger to grade >11, max 1 iteration.
Cold start (first request)	+5–10 s	~0 s	FastAPI lifespan warmup task pre-loads the model.
Total p50 latency	46.6 s	19.3 s	–58 %

Results and Discussion

End-to-End Output

For the canonical test input — *warfarin 5 mg, amiodarone 200 mg, ibuprofen 400 mg PRN, simvastatin 40 mg, clarithromycin 500 mg b.i.d.* — the system produced five medications correctly resolved to RxNorm codes with full confidence, seven interactions across three severity tiers (five high, one moderate, one low), and a plain-English summary at grade 10.7. The total pipeline ran in 19.3s end-to-end on the test hardware.

LLM-as-Judge Reliability

Across 17 test cases distributed over the four agents, the judge passed 14 (82% pass rate). The three failures clustered in the Coordinator suite, where the relaxation of the LLM sanity check (in favour of performance) means a small number of borderline inputs are no longer rejected upstream. We accept this trade-off because downstream agents gracefully return empty results in those cases.

Findings

F1. Tool-grounding (RxNorm + openFDA + curated DB) is necessary and sufficient to suppress LLM hallucinations on factual claims (RxCUIs, mechanisms, severities). The judge flagged zero fabricated RxCUIs across all test cases.

F2. Iterative self-correction (the Communicator’s `simplify_text` loop) is expensive at SLM

cost; a tightened trigger threshold and a single-iteration cap recover most of the latency without measurably degrading output quality.

F3. Custom JSONL tracing is sufficient for production observability and enables data-driven optimisation; we identified the two largest hot-spots in under five minutes of trace analysis.

Conclusions and Future Work

RxSentinel demonstrates that a four-agent multi-agent pipeline — previously the preserve of frontier-model deployments — is achievable on consumer hardware in 2026 with an open-source stack. The system delivers tool-grounded, plain-English medication safety reviews in ~ 19 s end-to-end, with full observability and explicit self-evaluation, and without any paid LLM API.

Promising extensions include (i) multilingual patient summaries (qwen2.5 already handles Sinhala and Tamil), (ii) pill-image OCR via a local vision model, (iii) token-level streaming of the patient summary to improve perceived latency, and (iv) a signed clinician-feedback endpoint feeding a domain-specific fine-tuning dataset.

Individual Contributions

Member	Built	GitHub
Jayasuriya L. K. R. S. (Rivin)	Coordinator agent + state validator + LangGraph wiring + tracing + FastAPI + SSE	ri7in
Piyarisi T. D. (Thusala)	Medication Parser agent + RxNorm tool + dose normalizer + parser eval suite	thusalapi
Wickramasooriya J. D. A. S. (Avishka)	Interaction Analyzer agent + interaction checker + severity rules + curated severe-pair dataset + analyzer eval suite	ashehxn
Manamperi S. A. (Sachila)	Patient Communicator agent + Flesch–Kincaid grader + simplifier loop + complete Next.js front-end	SAwandya

References

References

- [1] A. Bates et al., *Adverse drug events as a cause of hospital admission*, Br. J. Clin. Pharmacol., 2020.
- [2] National Library of Medicine, *RxNorm REST API documentation*, <https://rxnav.nlm.nih.gov/RxNormAPIs.html>.
- [3] U.S. Food and Drug Administration, *openFDA Drug Event Endpoint*, <https://open.fda.gov/apis/drug/event/>.
- [4] LangChain AI, *LangGraph documentation*, <https://langchain-ai.github.io/langgraph/>.
- [5] Ollama Inc., *Local model serving*, <https://ollama.com>.
- [6] J. P. Kincaid et al., *Derivation of New Readability Formulas for Navy Enlisted Personnel*, Naval Tech. Training Command, 1975.
- [7] Qwen Team, Alibaba Cloud, *Qwen2.5: A Party of Foundation Models*, 2024.

Source Code

The complete implementation, including all agents, tools, tests, the Next.js front-end, the LaTeX source for this report, and the demo video script, is publicly available at: <https://github.com/ri7in/ctse-assignment-2-rxsentinel>